

UNIDAD 4

Computacion Paralela y Distribuida

De la paralelizacion de tareas al procesamiento masivo de datos en la nube

Aplicaciones Distribuidas - ISR-701

Prof. Gleiston Guerrero Ulloa

Universidad Tecnica Estatal de Quevedo

Facultad de Ciencias de la Computacion - Septimo semestre - 2026-2027

Objetivos de aprendizaje de la unidad

1

Comprender

Distinguir paralelismo de datos y paralelismo de tareas, y elegir el modelo de programación apropiado (memoria compartida vs paso de mensajes).

2

Analizar

Aplicar la Ley de Amdahl y las métricas de speedup (aceleración), eficiencia y escalabilidad para justificar decisiones de diseño.

3

Comparar

Contrastar los modelos IaaS, PaaS, SaaS, FaaS y Edge Computing, y los proveedores AWS, Azure y GCP para casos reales.

4

Aplicar

Diseñar pipelines (tuberías de procesamiento) con MapReduce, Hadoop, Spark y Kafka Streams para datos masivos y streaming (flujo en tiempo real).

5

Evaluar

Elegir la combinación tecnológica adecuada para el Proyecto Fin de Curso según latencia, throughput (rendimiento) y costo.

4.1

Fundamentos de Computacion Paralela

- Paralelismo de datos vs tareas
- Memoria compartida vs paso de mensajes
- Ley de Amdahl y metricas
- OpenMP y MPI

4.2

Computacion en la Nube y Servicios Distribuidos

- IaaS, PaaS, SaaS
- AWS vs Azure vs GCP
- FaaS y serverless
- Grid computing y Edge computing

4.3

Procesamiento Distribuido de Datos

- Paradigma MapReduce
- Apache Hadoop
- Apache Spark
- Streaming con Kafka Streams

BLOQUE 4.1

Fundamentos de Computacion Paralela

Del hilo unico al calculo multi-nucleo y multi-nodo

Paralelismo (parallelism, ejecución simultánea): descomposición de un problema en subproblemas que se ejecutan al mismo tiempo en distintas unidades de procesamiento.



Paralelismo de datos

(data parallelism, mismo trabajo, distintos datos)

- El mismo programa se aplica sobre particiones distintas del conjunto de datos.
- Modelo SIMD (Single Instruction, Multiple Data - Instrucción Única, Datos Múltiples).
- Escala con el tamaño del dataset (conjunto de datos).

Ejemplo: multiplicar un vector de 10^9 elementos por un escalar; entrenamiento de redes neuronales con lotes.



Paralelismo de tareas

(task parallelism, distinto trabajo, mismos o distintos datos)

- Tareas diferentes se ejecutan concurrentemente, posiblemente sobre datos compartidos.
- Modelo MIMD (Multiple Instruction, Multiple Data - Instrucciones Múltiples, Datos Múltiples).
- Escala con la cantidad de tareas independientes disponibles.

Ejemplo: pipeline ETL (Extract, Transform, Load - Extraer, Transformar, Cargar) con extracción, limpieza y carga simultáneas.

Memoria compartida

(shared memory)

Multiples hilos acceden a un mismo espacio de direcciones de memoria dentro de un nodo.

Ventajas

- ▶ Comunicacion implicita: escribir en una variable la comparte.
- ▶ Baja latencia (nanosegundos).
- ▶ Programacion mas natural para el desarrollador.

Retos

- ▶ Race conditions (condiciones de carrera): requiere locks (cerrojos), mutex (mutual exclusion, exclusion mutua) o semaforos.
- ▶ No escala mas alla del nodo fisico (limite del bus de memoria).
- ▶ False sharing (falso compartido) degrada el rendimiento por el cache (memoria intermedia).

Herramientas tipicas: OpenMP, POSIX Threads (pthreads), Java Concurrency.

Paso de mensajes

(message passing)

Cada proceso tiene su propia memoria; se comunican intercambiando mensajes explicitos.

Ventajas

- ▶ Escala a miles de nodos en un cluster (agrupacion de computadoras).
- ▶ Sin problemas de coherencia de cache entre procesos.
- ▶ Portable a memoria distribuida y a nube.

Retos

- ▶ Comunicacion explicita: send (enviar) y receive (recibir) en el codigo.
- ▶ Alta latencia (microsegundos a milisegundos por red).
- ▶ Deadlock (bloqueo mutuo) si los patrones de envio-recepcion no calzan.

Herramientas tipicas: MPI (Message Passing Interface), Actor Model (Modelo de Actores), gRPC entre procesos.

Formula

$$S(N) = \frac{1}{(1 - P) + P / N}$$

S(N) = speedup (aceleracion) con N procesadores.

P = fraccion paralelizable del programa ($0 \leq P \leq 1$).

1 - P = fraccion estrictamente secuencial.

N = numero de procesadores o nucleos disponibles.

Limite teorico: $S(\text{inf}) = 1 / (1 - P)$

Un 5% secuencial impone speedup maximo de 20x, sin importar N.

Interpretacion practica

P (parte paralela)	N = 4	N = 16	N = 1024	Limite
50%	1,60	1,88	2,00	2x
80%	2,50	4,00	4,98	5x
95%	3,48	9,14	19,63	20x
99%	3,88	13,91	91,18	100x

Lectura pedagogica

- Optimizar la parte secuencial rinde mas que agregar procesadores.
- La comunicacion y la sincronizacion se descuentan de P.
- La Ley de Gustafson (Gustafson-Barsis) matiza: si el problema crece con N, se rompe el techo de Amdahl.
- En Spark y MapReduce la fase de shuffle (redistribucion) suele ser la mayor porcion secuencial percibida.



Speedup (aceleracion)

$$S(N) = T(1) / T(N)$$

Cuantas veces mas rapido corre el algoritmo con N procesadores respecto al secuencial. Ideal: $S(N) = N$ (aceleracion lineal).



Eficiencia

$$E(N) = S(N) / N$$

Fraccion del rendimiento maximo que aprovechamos por procesador. Valores entre 0 y 1; por debajo de 0,5 indica desperdicio significativo.



Throughput (rendimiento)

Trabajo util por unidad de tiempo

Cantidad de operaciones o transacciones por segundo. En sistemas de datos: MB/s procesados, eventos/s, jobs/hora.



Escalabilidad

Fuerte vs debil (strong vs weak scaling)

Fuerte: mantengo el problema, agrego nodos y mido T.
Debil: agrando el problema al ritmo de N y espero T constante. Sistemas cloud priorizan la debil.

OpenMP (Open Multi-Processing, Procesamiento Multiple Abierto): API (Application Programming Interface, Interfaz de Programacion de Aplicaciones) basada en directivas para C, C++ y Fortran, con paralelismo por hilos dentro de un nodo.

Características clave



Fork-Join

El hilo maestro crea (fork) equipo de hilos y los une (join) al terminar.



Directivas `#pragma omp`

El programador anota el código secuencial; el compilador genera el paralelo.



Regiones paralelas

parallel, for, sections, task, taskloop.



Clausulas de datos

shared, private, firstprivate, reduction, atomic, critical.



Sincronizacion

barrier (barrera), critical (seccion critica), atomic, ordered.

Ejemplo: suma paralela con reduction

```
#include <omp.h>
#include <stdio.h>

double sum = 0.0;
int N = 1000000;

#pragma omp parallel for \
    reduction(+:sum)
for (int i = 0; i < N; i++) {
    sum += f(i);
}

printf("Total: %f\n", sum);
```

Compilacion: `gcc -fopenmp suma.c -o suma`

MPI (Message Passing Interface, Interfaz de Paso de Mensajes): estandar portable para comunicacion entre procesos que se ejecutan en el mismo nodo o en nodos distintos de un cluster (agrupacion de computadoras).

Primitivas fundamentales



MPI_Init / MPI_Finalize

Inicializa y libera el entorno paralelo.



MPI_Comm_size / MPI_Comm_rank

Numero total de procesos y rango (rank, identificador) del proceso actual.



MPI_Send / MPI_Recv

Comunicacion punto a punto (P2P, peer to peer, entre pares) bloqueante.



MPI_Bcast / MPI_Scatter / MPI_Gather

Comunicaciones colectivas: broadcast (difusion), scatter (dispersion), gather (recoleccion).



MPI_Reduce / MPI_Allreduce

Reduccion global (suma, max, min, and, or) sobre todos los procesos.

Ejemplo: hola desde cada proceso

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD,
                  &rank);
    MPI_Comm_size(MPI_COMM_WORLD,
                  &size);
    printf("Proceso %d de %d\n",
          rank, size);
    MPI_Finalize();
    return 0;
}
```

```
mpicc hola.c -o hola | mpirun -np 4 ./hola
```

BLOQUE 4.2

Computacion en la Nube y Servicios Distribuidos

Del datacenter propio al pago por milisegundo de ejecucion

4.2.1 Modelos de servicio en la nube

IaaS

Infrastructure as a Service (Infraestructura como Servicio)

El proveedor entrega recursos virtualizados: VM (Virtual Machine, Maquina Virtual), red, almacenamiento en bloque.

Cliente gestiona: SO (Sistema Operativo), runtime, aplicaciones, datos.

Ejemplos: EC2 (Elastic Compute Cloud), Compute Engine, Azure VM.

Maxima flexibilidad y control.

PaaS

Platform as a Service (Plataforma como Servicio)

El proveedor entrega la plataforma completa: SO, runtime, middleware, DB gestionada.

Cliente gestiona: Aplicaciones y datos.

Ejemplos: App Engine, Heroku, Elastic Beanstalk, Azure App Service.

Foco en el código, no en la infraestructura.

SaaS

Software as a Service (Software como Servicio)

El proveedor entrega la aplicación terminada, accesible por navegador o API.

Cliente gestiona: Sus datos y su configuración.

Ejemplos: Gmail, Salesforce, Microsoft 365, Google Workspace.

Cero mantenimiento técnico para el usuario final.

4.2.2 Comparacion: AWS, Azure y GCP

Dimension	AWS (Amazon Web Services)	Microsoft Azure	Google Cloud (GCP)
Fortaleza	Amplitud de servicios y madurez del ecosistema	Integracion con stack Microsoft y entorno empresarial	Big data (datos masivos), IA y Kubernetes
Computo (IaaS)	EC2, Lightsail	Azure VM, Virtual Machine Scale Sets	Compute Engine, GKE (Google Kubernetes Engine)
Serverless (FaaS)	AWS Lambda	Azure Functions	Cloud Functions, Cloud Run
Almacenamiento	S3 (Simple Storage Service), EBS	Blob Storage, Managed Disks	Cloud Storage, Persistent Disk
Bases de datos	RDS, DynamoDB, Aurora	SQL Database, Cosmos DB	Cloud SQL, Spanner, Bigtable, Firestore
Analitica / Big data	EMR, Redshift, Athena, Kinesis	Synapse Analytics, HDInsight, Stream Analytics	Dataproc, BigQuery, Dataflow, Pub/Sub
Ideal para	Startups a corporativos, cargas variadas y globales	Empresas con Windows Server, Active Directory, .NET	Analitica avanzada, ML (Machine Learning) y contenedores

4.2.3 FaaS y computacion serverless

FaaS (Function as a Service, Funcion como Servicio): modelo serverless (sin servidor visible) donde el proveedor ejecuta funciones cortas en respuesta a eventos, cobrando por milisegundo de uso y por invocacion, sin que el cliente aprovisione ni escale servidores.

Caracteristicas del modelo

- **Event-driven (dirigido por eventos)**
HTTP, mensajes de cola, cambios en base de datos, subida de archivos.
- **Escalado automatico a cero**
De cero a miles de instancias en segundos; sin trafico, sin costo.
- **Pago por uso (pay-per-use)**
Cobro por invocacion + tiempo de ejecucion, no por hora reservada.
- **Stateless (sin estado)**
El estado se guarda fuera: cache (Redis), DB, cola, objeto en S3.
- **Cold start (arranque en frio)**
Primera invocacion suma latencia de inicializacion del runtime.

Casos de uso ideales

- APIs REST de baja concurrencia y trafico esporadico.
- Procesamiento de imagenes y videos al subir archivos.
- Cron jobs (tareas programadas) y ETL ligeros.
- Webhooks y chatbots.
- Backend para logica de IoT (Internet of Things, Internet de las Cosas).

Limites y precauciones

- Tiempo maximo por ejecucion (por ejemplo, 15 min en AWS Lambda).
- Cold start puede afectar SLA (Service Level Agreement, Acuerdo de Nivel de Servicio) en APIs criticas.
- Depuracion mas dificil que en un servidor tradicional.
- Costos altos para cargas sostenidas y CPU-intensivas (mejor un contenedor).

4.2.4 Computacion en malla y computacion en el borde

Grid computing

(computacion en malla o en cuadrricula)

Federacion de recursos heterogeneos, geograficamente dispersos y administrados por organizaciones distintas, orquestados como una unica plataforma virtual.

- Origen cientifico: LHC-Grid (CERN), SETI@home, BOINC (Berkeley Open Infrastructure for Network Computing).
- Uso oportunista de ciclos de CPU inactivos.
- Autonomia administrativa: cada nodo mantiene su politica local.
- Alta heterogeneidad: SO, hardware y redes distintas.
- Estandares: OGSA (Open Grid Services Architecture), Globus Toolkit.

Hoy: casos residuales en investigacion; muchos casos migraron a HPC-cloud y a Kubernetes federado.

Edge computing

(computacion en el borde de la red)

Procesamiento cerca de la fuente de datos (dispositivos IoT, sensores, camaras, moviles) para reducir latencia y consumo de ancho de banda hacia la nube central.

- Latencia baja: milisegundos, indispensable para vehiculos autonomos, salud remota y AR/VR (Realidad Aumentada / Realidad Virtual).
- Ahorro de red: solo se envia lo relevante o agregado.
- Privacidad: datos sensibles pueden no salir del sitio (edge medico).
- Fog computing (computacion en niebla): capa intermedia entre edge y nube.
- Plataformas: AWS Greengrass, Azure IoT Edge, Google Distributed Cloud Edge.

Hoy: motor del 5G MEC (Multi-access Edge Computing) y de la IA embebida en camaras industriales.

BLOQUE 4.3

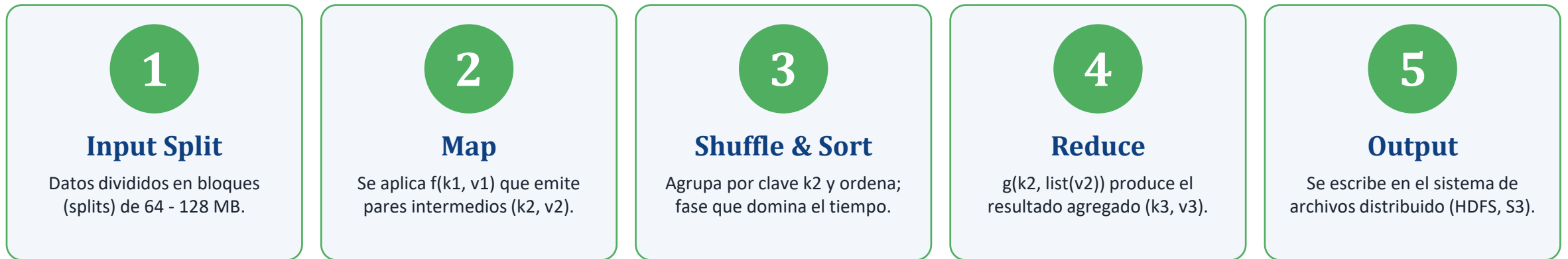
Procesamiento Distribuido de Datos

De MapReduce al streaming en tiempo real

4.3.1 Paradigma MapReduce

MapReduce (mapear y reducir): modelo de programación propuesto por Google (2004) para procesar terabytes o petabytes de datos en clusters de miles de computadoras commodity (económicas). El programador escribe dos funciones puras: **map** y **reduce**; el framework se encarga del resto.

Flujo del procesamiento



WordCount: el Hola Mundo de MapReduce

```
map(linea)      -> for each palabra in linea: emit(palabra, 1)
reduce(palabra, lista_de_unos) -> emit(palabra, sum(lista_de_unos))
```

Ganancia clave: paralelismo de datos automatico + tolerancia a fallos por reintento de tareas.

4.3.2 Apache Hadoop: ecosistema y arquitectura

Apache Hadoop: framework open source (codigo abierto) inspirado en los papers de Google (GFS y MapReduce) para almacenar y procesar datos masivos en clusters de hardware commodity (basico).

Arquitectura en 3 capas

HDFS (Hadoop Distributed File System)

Sistema de Archivos Distribuido de Hadoop

NameNode + DataNodes; bloques de 128 MB replicados 3 veces por defecto.

YARN (Yet Another Resource Negotiator)

Otro Negociador de Recursos Mas

ResourceManager + NodeManagers; gestiona CPU y RAM del cluster.

MapReduce / Tez / Spark

Motor de procesamiento

MapReduce clasico o motores mas rapidos como Tez y Spark sobre YARN.

Todo sobre commodity hardware y tolerante a fallos

Ecosistema Hadoop

Hive

SQL sobre Hadoop, para analistas familiarizados con SQL.

Pig

Lenguaje de scripting (Pig Latin) para pipelines de datos.

HBase

Base de datos NoSQL columnar sobre HDFS (estilo BigTable).

Sqoop

Transferencia de datos entre HDFS y bases relacionales.

Flume

Ingesta de logs (registros) hacia HDFS en streaming.

Oozie

Orquestador de workflows (flujos de trabajo) sobre YARN.

ZooKeeper

Coordinacion distribuida (elecciones de lider, locks).

4.3.3 Apache Spark: procesamiento en memoria

Apache Spark: motor unificado para procesamiento distribuido de datos que trabaja principalmente *in-memory (en memoria)*, 10 a 100 veces mas rapido que MapReduce clasico para cargas iterativas. Origen: UC Berkeley AMPLab, 2010.

Modelo de ejecucion



RDD (Resilient Distributed Dataset)

Conjunto de Datos Distribuido Resiliente: coleccion inmutable, particionada, con linaje para recomputar en fallos.



DAG (Directed Acyclic Graph)

Grafo Aciclico Dirigido de transformaciones; el optimizador Catalyst decide el mejor plan.



Transformaciones vs acciones

Lazy: transformaciones (map, filter, join) construyen el DAG; solo las acciones (count, collect, save) disparan la ejecucion.



DataFrame / Dataset / Spark SQL

APIs de alto nivel con esquema, mas eficientes que RDD para casos analiticos.

Modulos del ecosistema

Spark SQL - SQL + DataFrame

Structured Streaming - Streaming continuo

MLlib - Machine Learning

GraphX - Procesamiento de grafos

PySpark: WordCount con DataFrames

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split

spark = SparkSession.builder.getOrCreate()
lineas = spark.read.text("hdfs://quijote.txt")
conteo = (lineas
    .select(explode(split("value", " ")).alias("w"))
    .groupBy("w").count()
    .orderBy("count", ascending=False))
conteo.show(10)
```

4.3.4 Streaming en tiempo real con Kafka Streams

Apache Kafka: plataforma distribuida de streaming basada en un log (bitacora) inmutable y particionado. **Kafka Streams:** libreria (biblioteca de programacion) Java/Scala para construir aplicaciones que consumen, transforman y publican streams (flujos de eventos) en tiempo real, con procesamiento exactly-once (exactamente una vez).

Abstracciones clave

Topic (topico)

Categoria de mensajes, dividida en particiones ordenadas.

Producer / Consumer

Productores publican eventos; consumidores los leen a su ritmo.

KStream

Stream de eventos (cada evento se procesa una vez).

KTable

Tabla mutable derivada de un stream (ultimo valor por clave).

Windowing

Ventanas de tiempo: tumbling (fija), hopping (deslizante), session.

Exactly-once semantics

Semantica de exactamente una vez para escrituras y lecturas.

Patrones tipicos

- Deteccion de fraude en pagos en tiempo real.
- Enriquecimiento de eventos con join de stream + tabla.
- Agregaciones por ventana (transacciones por minuto).
- CDC (Change Data Capture, Captura de Cambios de Datos) desde bases operativas.

Ejemplo minimo (Java)

```
StreamsBuilder builder = new StreamsBuilder();
KStream<String, Pago> pagos =
    builder.stream("pagos");

pagos.filter((k, v) -> v.getMonto() > 5000)
    .mapValues(v -> enriquecer(v))
    .to("pagos-sospechosos");

new KafkaStreams(builder.build(), props)
    .start();
```

Escenario	Tecnologia recomendada	Por que
Simulacion cientifica CPU-intensiva en 1 nodo	OpenMP (memoria compartida)	Directivas simples; sin transferencia por red.
Simulacion en cluster de decenas de nodos	MPI (paso de mensajes) + OpenMP hibrido	Escala HPC; control fino de comunicaciones.
API con trafico esporadico y estacional	FaaS (Lambda, Cloud Functions)	Escalado a cero; pago por invocacion.
Backend estable con carga sostenida	Contenedores en PaaS o Kubernetes	Mejor costo/hora que FaaS a alta concurrencia.
Vehiculos autonomos y camaras IoT	Edge computing + 5G MEC	Latencia < 10 ms; privacidad en el sitio.
ETL nocturno sobre terabytes historicos	Spark en Hadoop / EMR / Databricks	Optimo para batch (por lotes) y ML iterativo.
Deteccion de fraude en pagos	Kafka + Kafka Streams	Streaming exactly-once, latencias sub-segundo.

Glosario Nemonicos y terminos (1 de 2): paralelismo y nube

API (*Application Programming Interface*) - Interfaz de Programacion de Aplicaciones.

CPU (*Central Processing Unit*) - Unidad Central de Procesamiento.

GPU (*Graphics Processing Unit*) - Unidad de Procesamiento Grafico.

SIMD (*Single Instruction, Multiple Data*) - Instruccion Unica, Datos Multiples.

MIMD (*Multiple Instruction, Multiple Data*) - Instrucciones Multiples, Datos Multiples.

SPMD (*Single Program, Multiple Data*) - Programa Unico, Datos Multiples.

OpenMP (*Open Multi-Processing*) - Procesamiento Multiple Abierto (memoria compartida).

MPI (*Message Passing Interface*) - Interfaz de Paso de Mensajes.

HPC (*High Performance Computing*) - Computo de Alto Rendimiento.

Speedup (*Aceleracion*) - $S(N) = T(1)/T(N)$; cuanto mas rapido corre con N procesadores.

Throughput (*Rendimiento*) - Trabajo util por unidad de tiempo.

IaaS (*Infrastructure as a Service*) - Infraestructura como Servicio.

PaaS (*Platform as a Service*) - Plataforma como Servicio.

SaaS (*Software as a Service*) - Software como Servicio.

FaaS (*Function as a Service*) - Funcion como Servicio (serverless).

VM (*Virtual Machine*) - Maquina Virtual.

SLA (*Service Level Agreement*) - Acuerdo de Nivel de Servicio.

AWS (*Amazon Web Services*) - Servicios Web de Amazon.

GCP (*Google Cloud Platform*) - Plataforma de Nube de Google.

MEC (*Multi-access Edge Computing*) - Computacion en el Borde Multi-Acceso (5G).

IoT (*Internet of Things*) - Internet de las Cosas.

CDN (*Content Delivery Network*) - Red de Distribucion de Contenido.

Nemonicos y terminos (2 de 2): datos distribuidos

HDFS (*Hadoop Distributed File System*) - Sistema de Archivos Distribuido de Hadoop.

YARN (*Yet Another Resource Negotiator*) - Otro Negociador de Recursos Mas (gestor de recursos de Hadoop).

GFS (*Google File System*) - Sistema de Archivos de Google (inspiro a HDFS).

RDD (*Resilient Distributed Dataset*) - Conjunto de Datos Distribuido Resiliente (Spark).

DAG (*Directed Acyclic Graph*) - Grafo Aciclico Dirigido (plan de ejecucion Spark).

ETL (*Extract, Transform, Load*) - Extraer, Transformar, Cargar (pipeline clasico).

ELT (*Extract, Load, Transform*) - Extraer, Cargar, Transformar (patron moderno cloud).

CDC (*Change Data Capture*) - Captura de Cambios de Datos (para replicacion en streaming).

MapReduce (*Mappear y Reducir*) - Paradigma de Google (2004) para procesar datos masivos.

KStream (*Kafka Stream*) - Flujo de eventos en Kafka Streams.

KTable (*Kafka Table*) - Tabla mutable derivada de un stream (ultimo valor por clave).

P2P (*Peer to Peer*) - Entre pares (comunicacion punto a punto).

Shuffle (*Redistribucion*) - Reparto de datos por clave entre nodos; suele ser el cuello de botella.

Lazy eval. (*Evaluacion perezosa*) - Las transformaciones no se ejecutan hasta una accion.

Exactly-once (*Exactamente una vez*) - Garantia de que cada evento se procesa una unica vez.

MLlib (*Machine Learning library*) - Biblioteca de aprendizaje automatico de Spark.

- [1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities", en Proc. AFIPS Spring Joint Computer Conf., 1967, pp. 483-485.
- [2] J. Dean y S. Ghemawat, "MapReduce: simplified data processing on large clusters", Commun. ACM, vol. 51, num. 1, pp. 107-113, ene. 2008. doi: 10.1145/1327452.1327492.
- [3] M. Zaharia et al., "Apache Spark: a unified engine for big data processing", Commun. ACM, vol. 59, num. 11, pp. 56-65, oct. 2016. doi: 10.1145/2934664.
- [4] S. Ghemawat, H. Gobioff y S.-T. Leung, "The Google file system", en Proc. 19th ACM Symp. Operating Systems Principles (SOSP), 2003, pp. 29-43. doi: 10.1145/945445.945450.
- [5] P. Mell y T. Grance, "The NIST definition of cloud computing", NIST Special Publication 800-145, sep. 2011. doi: 10.6028/NIST.SP.800-145.
- [6] B. Chapman, G. Jost y R. van der Pas, Using OpenMP: Portable Shared Memory Parallel Programming. Cambridge, MA: MIT Press, 2007.
- [7] W. Gropp, E. Lusk y A. Skjellum, Using MPI: Portable Parallel Programming with the Message-Passing Interface, 3ra ed. Cambridge, MA: MIT Press, 2014.
- [8] I. Baldini et al., "Serverless computing: current trends and open problems", en Research Advances in Cloud Computing, S. Chaudhary et al., Eds. Singapur: Springer, 2017, pp. 1-20. doi: 10.1007/978-981-10-5026-8_1.
- [9] W. Shi, J. Cao, Q. Zhang, Y. Li y L. Xu, "Edge computing: vision and challenges", IEEE Internet of Things Journal, vol. 3, num. 5, pp. 637-646, oct. 2016. doi: 10.1109/JIOT.2016.2579198.
- [10] J. Kreps, N. Narkhede y J. Rao, "Kafka: a distributed messaging system for log processing", en Proc. 6th Int. Workshop on Networking Meets Databases (NetDB), Atenas, Grecia, jun. 2011.
- [11] G. Wang et al., "Building a replicated logging system with Apache Kafka", Proc. VLDB Endow., vol. 8, num. 12, pp. 1654-1655, ago. 2015. doi: 10.14778/2824032.2824063.

Gracias.

Preguntas, dudas y proximos pasos

Prof. Gleiston Guerrero Ulloa - gguerrero@uteq.edu.ec

Aplicaciones Distribuidas (ISR-701) - UTEQ - Facultad de Ciencias de la Computacion

Recordatorio: la Entrega 4 aplica estos conceptos de forma integradora.